

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
```

```
class GopichEmotionModel(nn.Module):
    def __init__(self, input_dim=10,
memory_dim=32, emotion_dim=8):
        super().__init__()
```

```
        # 1. Импликативная память
(подсознательная)
```

```
        self.implicit_memory =
nn.LSTM(input_dim, memory_dim,
batch_first=True)
```

```
        # 2. Гормональный отклик
(эмоциональная реакция)
```

```
        self.hormonal_response = nn.Sequential(
            nn.Linear(memory_dim, emotion_dim),
            nn.Sigmoid() # [0,1] - интенсивность
ЭМОЦИЙ
```

)

3. Когнитивный отклик (осознание)

```
self.cognitive_path =  
nn.Linear(memory_dim + emotion_dim,  
memory_dim)
```

4. Экспликативная память

(сознательная)

```
self.explicit_memory =  
nn.LSTM(memory_dim, memory_dim,  
batch_first=True)
```

5. Чувство знания (метапамять)

```
self.knowledge_feeling =  
nn.Linear(memory_dim, 1)
```

6. Поведение (быстрое/медленное)

```
self.fast_behavior =  
nn.Linear(emotion_dim, 2) #
```

Эмоциональное

```
self.slow_behavior =
```

```
nn.Linear(memory_dim, 2) # Осознанное
```

```
def forward(self, x, external_info=None):
```

```
    # Шаг 1: Импликативное
```

```
    распознавание
```

```
    implicit_h, _ = self.implicit_memory(x)
```

```
    last_implicit = implicit_h[:, -1, :]
```

```
    # Шаг 2: Гормональный отклик
```

```
    (быстрая эмоция)
```

```
    emotion =
```

```
    self.hormonal_response(last_implicit)
```

```
    # Шаг 3: Два пути
```

```
    fast_path = emotion # Эмоциональный  
    отклик → быстрое поведение
```

```
    # Когнитивный путь (медленный)
```

```
    cognitive_input = torch.cat([last_implicit,  
    emotion], dim=-1)
```

```
    cognitive_state =
```

```
    F.relu(self.cognitive_path(cognitive_input))
```

Шаг 4: Экспликативная обработка

if external_info is not None:

 cognitive_input =

torch.cat([cognitive_state.unsqueeze(1),
external_info], dim=1)

else:

 cognitive_input =

cognitive_state.unsqueeze(1)

explicit_h, _ =

self.explicit_memory(cognitive_input)

last_explicit = explicit_h[:, -1, :]

Шаг 5: Чувство знания

feeling_knowledge =

torch.sigmoid(self.knowledge_feeling(last_explicit))

Шаг 6: Поведение

fast_action =

F.softmax(self.fast_behavior(fast_path),

```
dim=-1)
    slow_action =
F.softmax(self.slow_behavior(last_explicit),
dim=-1)

    return {
        'implicit_state': last_implicit,
        'emotion': emotion,
        'cognitive_state': cognitive_state,
        'explicit_state': last_explicit,
        'feeling_knowledge':
feeling_knowledge,
        'fast_behavior': fast_action,
        'slow_behavior': slow_action
    }
```

Обучение

```
def train_gopich_model():
    model = GopichEmotionModel()
    optimizer =
optim.Adam(model.parameters(), lr=0.001)
    criterion = nn.MSELoss()
```

```
# Синтетические данные: стимул →  
желаемое поведение  
for epoch in range(100):  
    batch_size, seq_len = 32, 10  
    x = torch.randn(batch_size, seq_len, 10)  
# Стимулы  
    external = torch.randn(batch_size, 1, 32)  
# Внешняя информация  
  
    target_fast = torch.randn(batch_size, 2)  
    target_slow = torch.randn(batch_size, 2)  
  
    out = model(x, external)  
  
    loss_fast = criterion(out['fast_behavior'],  
target_fast)  
    loss_slow =  
criterion(out['slow_behavior'], target_slow)  
    loss_knowledge =  
criterion(out['feeling_knowledge'],  
torch.ones(batch_size, 1) *
```

0.7)

```
    loss = loss_fast + loss_slow + 0.1 *  
loss_knowledge  
    optimizer.zero_grad()  
    loss.backward()  
    optimizer.step()  
  
    if epoch % 20 == 0:  
        print(f"Epoch {epoch}, Loss:  
{loss.item():.4f}")  
  
if __name__ == "__main__":  
    train_gopich_model()
```